

Problem 2 Full Report: OpenMP Acceleration of NEP Potential Calculation

June 20, 2026

1 Objective and Requirements

Problem 2 requires OpenMP acceleration of molecular-dynamics potential calculation. The work includes parallel-opportunity analysis, hotspot identification, correct treatment of shared data, performance testing with different thread counts, serial/parallel correctness comparison, and unit testing. This work focuses on the NEP CPU implementation in `source/source_md/potential/ml/nep`.

2 Parallel Analysis

The NEP calculation contains neighbor-list construction, descriptor and neural network derivative calculation, and radial/angular/ZBL force accumulation. The first two stages already contain atom-level OpenMP loops. The force stage was the main unfinished hotspot. For center atom n_1 and neighbor n_2 :

$$F_{n_1} += f_{12}, \quad F_{n_2} -= f_{12}.$$

Virial is also accumulated by atom index. A direct parallel loop therefore has write/write conflicts when threads update the same neighbor.

3 Three NEP Force Versions

The NEP API now exposes `NEP::ForceParallelMode` and `set_force_parallel_mode`. The default remains `ThreadLocal`, preserving optimized behavior for existing callers.

3.1 Version 1: Serial Force Baseline

`ForceParallelMode::Serial` executes radial, angular, and ZBL force loops serially. It provides the deterministic reference. Neighbor and descriptor stages may still use OpenMP, so additional threads can produce limited speedup outside force accumulation.

3.2 Version 2: OpenMP Atomic Accumulation

`ForceParallelMode::Atomic` parallelizes the outer atom loop and uses `omp atomic update` for every shared force, virial, and ZBL-energy addition. It uses little extra memory, but synchronization occurs repeatedly inside the neighbor loop.

3.3 Version 3: OpenMP Thread-Local Reduction

`ForceParallelMode::ThreadLocal` gives each thread private force and virial arrays; ZBL also receives private energy storage. Threads accumulate without synchronization, then reduce their buffers once after the loop. Its memory cost is $O(TN)$, but it avoids atomics in the hotspot.

All versions share the same force mathematics; only accumulation changes. The radial, angular, and ZBL small-box force kernels support all three modes.

4 Correctness Testing

The test `source/source_md/test/nep_omp_test.cpp` loads `tests/PP_ORB/nep_hfo2.txt`. It evaluates the same Hf/O system with serial, four-thread atomic, and four-thread thread-local modes. Potential, force, and virial use tolerances 10^{-12} , 10^{-10} , and 10^{-10} .

The sweep also compares every result with one-thread serial output. Across all 27 cases:

$$\max |\Delta E| = 0, \quad \max |\Delta F| = 1.83187 \times 10^{-15}, \quad \max |\Delta V| = 3.55271 \times 10^{-14}.$$

These are floating-point summation-order differences and are well below the test tolerances.

5 Benchmark Script

The benchmark consists of:

```
source/source_md/tools/nep_omp_sweep.cpp
source/source_md/tools/run_problem2_nep_sweep.sh
```

It generates deterministic alternating Hf/O cubic systems, compiles with `-O2 -fopenmp`, runs all modes with 1, 2, and 4 threads, checks each result, and writes CSV. The command used was:

```
cd source/source_md
tools/run_problem2_nep_sweep.sh ../tests/PP_ORB/nep_hfo2.txt \
  64,216,512 1,2,4 20 reports/problem2_nep_sweep_results.csv
```

Model loading is excluded from timing. Full raw results are stored in `reports/problem2_nep_sweep_results.csv`.

6 Performance Results

Speedup is relative to serial-force, one-thread execution for each size.

Atoms	Mode	1 thread (ms)	2 threads (ms)	4 threads (ms)	4-thread speedup
64	Serial	3.3298	2.7041	2.2777	1.46
64	Atomic	3.3062	2.5729	2.0893	1.59
64	Thread-local	3.2625	1.8249	1.2259	2.72
216	Serial	10.3443	8.2511	7.3411	1.41
216	Atomic	10.5334	6.7192	5.0961	2.03
216	Thread-local	10.1750	5.4566	2.8917	3.58
512	Serial	30.1092	23.1361	20.4003	1.48
512	Atomic	44.0341	19.8320	16.7979	1.79
512	Thread-local	27.4264	16.0081	10.0029	3.01

Table 1: Comparison of three NEP force-accumulation versions.

7 Analysis

The serial-force version scales modestly because neighbor and descriptor work is parallel while force remains a bottleneck. Its four-thread speedup stays below $1.5\times$. Atomic mode improves

scaling but shared-array synchronization limits it; its 512-atom one-thread result also shows atomic instruction overhead without contention.

Thread-local reduction is fastest overall. At four threads it reaches $2.72\times$, $3.58\times$, and $3.01\times$ speedup for 64, 216, and 512 atoms. Moving synchronization outside the neighbor loop is therefore more effective than atomic updates for every pair contribution.

8 Requirement Coverage and Conclusion

This implementation identifies dependencies and hotspots; parallelizes radial, angular, and ZBL force calculations; provides serial, atomic, and thread-local versions; handles races through atomics or private reduction; verifies numerical consistency; benchmarks three sizes and thread counts; and supplies a repeatable script and CSV.

Thread-local reduction is recommended as the default. Atomic mode provides a low-memory comparison, and serial mode remains the correctness baseline. Future work may reduce private-buffer memory using atom blocking or domain decomposition and test scaling beyond four cores.